CS673

Software Engineering

Spring 2025

Lecture 1

Ron Czik

Administrivia

- Grader: Camille Christie, camchris@bu.edu
  - Zoom office hours, Wednesday, 4:30 – 5:30, CAS 310
  - https://us05web.zoom.us/j/3283794868?pwd=OVVWeW5ycklIdURvdUNOa3B1bTVOUT09
- Meet Wednesdays, CAS B06B, 6:00 – 8:45 pm
- Office hours: Mondays 5:30 -6:30 pm CDS 1001
- Attendance, collaboration, and participation are mandatory
  - https://piazza.com/bu/spring2025/cs673a1 Access Code: 1nem4dhet4t
  - Small amount of extra credit for those who post often and provide good answers
- Assignments to be submitted on Gradescope
  - https://www.gradescope.com/courses/959499 Entry Code: DK7DEZ
  - No makups
- Syllabus

What we'll cover in this course

- Software Development Life Cycle (SDLC) such as Agile and DevOps
- Requirements analysis
- Software design and architecture
- Programming techniques
- Cloud computing
- Refactoring
- Testing
- AI in software engineering
  - Is he REALLY going to let us use AI for this class?
  - Yes, but correctly!

What we'll do tonight

- Introduction
- Administrivia/course overview
- Software products vs projects
- Product management
- Product prototyping
- Begin to think about teams for semester product

This course isn't for everyone

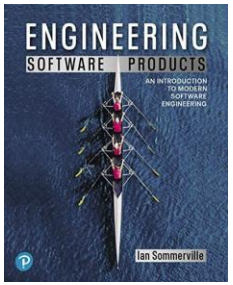
- This is a 500 level, experiential course
  - **LOTS** of programming – everyone must contribute (develop and debug) code
  - We will model industry development practices (Agile/Scrum)
  - Expectations are high and you're required to collaborate
- Acceptable languages are Java, Python, Kotlin
- This course isn't for you if
  - You don't have time
  - Not able to work in groups
  - Don't have a strong programming background (control structures, looping, arrays, object-oriented techniques)

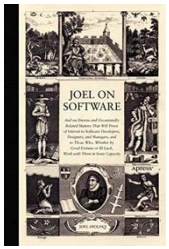
Teamwork IS for everyone

- You will all be working together, very closely
- You will be expected to meet and collaborate regularly (several times per week outside of class)
- You are not allowed to coast and receive the same grade as your teammates
  - If you do, you will be graded separately

Textbook and resources

- Engineering Software Products – REQUIRED
- Software Engineering – OPTIONAL
- Joel On Software – OPTIONAL
- Head First Design Patterns – OPTIONAL
- Design Patterns Elements of Reusable Object-Oriented Software – OPTIONAL
- Others





Commit!

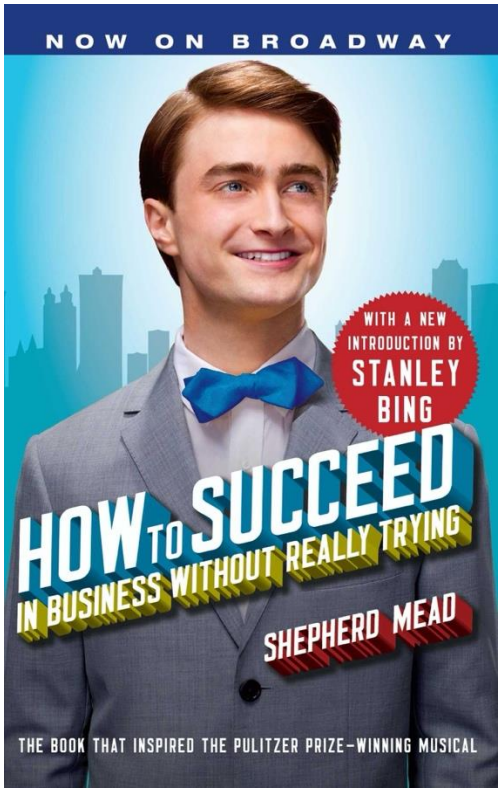
- Be ready to commit significant effort and time to this course
  - LOTS of new material
  - LOTS of independent research – there is a lot to discuss, and we can't cover all of it in 2.75 hours of class time each week

- Be considerate
  - Do not use electronics while I'm lecturing or while others are presenting – that's just rude!
  - It'll also effect your grade
  - Pay attention, be engaged

- Be accommodating
  - Work closely with your teammates
  - Be flexible, polite, and helpful
- Have high expectations of yourself and your teammates
  - No coasting
  - Everyone contributes – if not, see me
  - We will be checking

How to succeed?

- Come class prepared every week
  - Read and review the book
  - Be on time
- Be helpful – share ideas
- Listen!
- Meet regularly outside of class – at least twice a week
- Be nice
- Work hard



Introduction and ice breakers

- About me
  - I'm the director of technology and digital strategy at the MIT Museum
  - BA in astronomy and MS in computer science, both from BU
  - I've been in the SW industry for 40 years, working at large and small companies
  - I bring the real-world experience into the classroom
  - Education philosophy
  - Collaboration and ongoing assignments – not testing
  - Collaboration on homework assignments isn't cheating – if everyone contributes

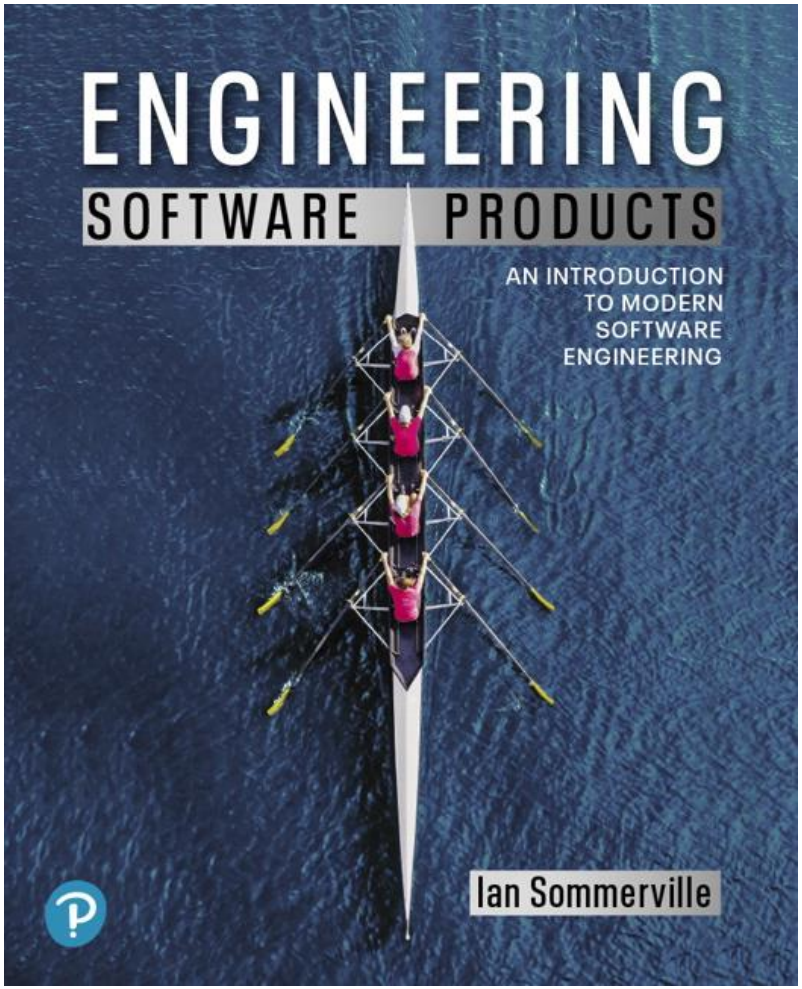
Ice breaker

- About you
  - Your name

  And (pick one)
  - One thing you wish you knew how to do or one thing you recently learned how to do
  - One thing that's interesting about you
  - What you hope to get from this course
  - Any project ideas you have already

Engineering Software Products

First Edition



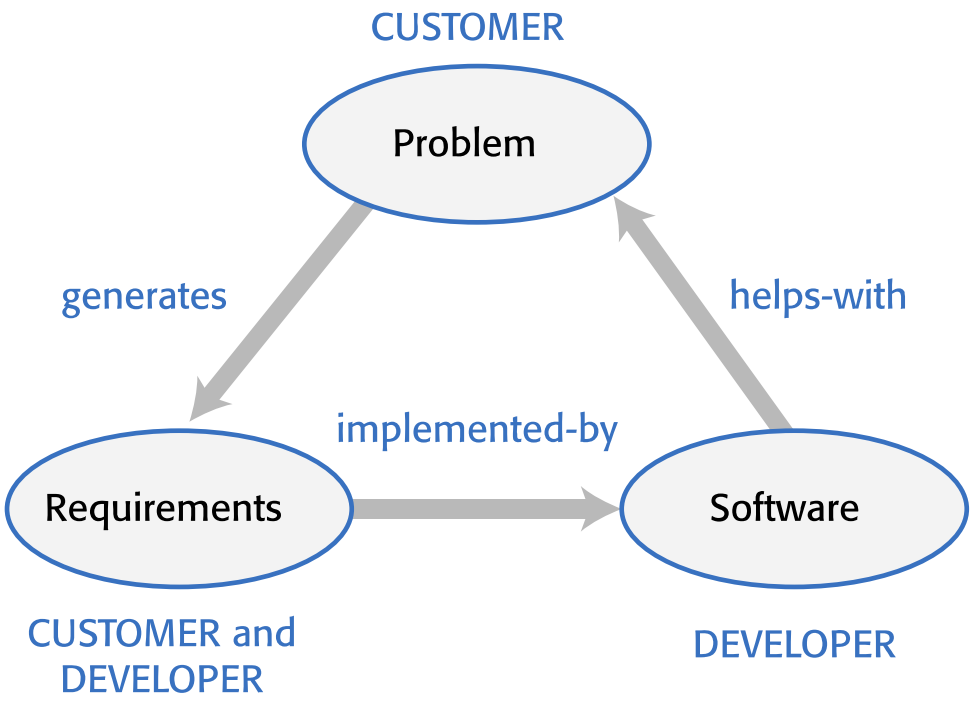
Chapter 1

Software products

Software products

- Software products are generic software systems that provide functionality that is useful to a range of customers.
- Many different types of products are available from large-scale business systems (e.g. MS Excel) through personal products (e.g. Evernote) to simple mobile phone apps and games (e.g. Suduko).
- Software product engineering methods and techniques have evolved from software engineering techniques that support the development of one-off, custom software systems.
- Custom software systems are still important for large businesses, government and public bodies. They are developed in dedicated software projects.

Figure 1.1



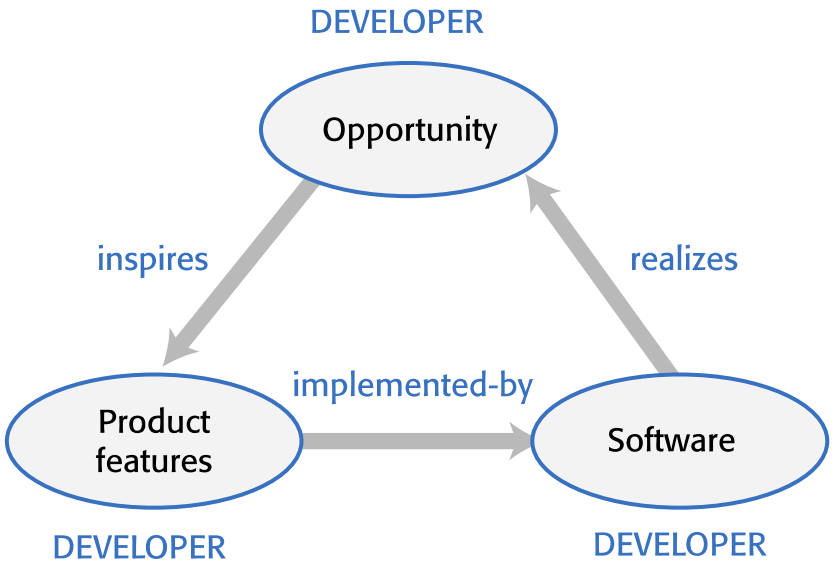
Project-based software engineering (1 of 2)

- The starting point for the software development is a set of ‘software requirements’ that are owned by an external client and which set out what they want a software system to do to support their business processes.
- The software is developed by a software company (the contractor) who design and implement a system that delivers functionality to meet the requirements.

Project-based software engineering (2 of 2)

- The customer may change the requirements at any time in response to business changes (they usually do). The contractor must change the software to reflect these requirements changes.
- Custom software usually has a long-lifetime (10 years or more) and it must be supported over that lifetime.

Figure 1.2



Product-based software engineering

Product software engineering (1 of 2)

- The starting point for product development is a business opportunity that is identified by individuals or a company. They develop a software product to take advantage of this opportunity and sell this to customers.
- The company who identified the opportunity design and implement a set of software features that realize the opportunity and that will be useful to customers.

Product software engineering (2 of 2)

- The software development company are responsible for deciding on the development timescale, what features to include and when the product should change.
- Rapid delivery of software products is essential to capture the market for that type of product.

Table 1.1 Software product lines and platforms (1 of 2)

Technology	Description
Software product line	Software product line is a set of software products that share a common core. Each member of the product line includes customer-specific adaptations and additions. Software product lines may be used to implement a custom system for a customer with specific needs that can't be met by a generic product. For example, a company providing communication software to the emergency services may have a software product line where the core product includes basic communication services such as receive and log calls, initiate an emergency response, pass information to vehicles, and so on. However, each customer may use different radio equipment, and their vehicles may be equipped in different ways. The core product has to be adapted for each customer to work with the equipment that they use.

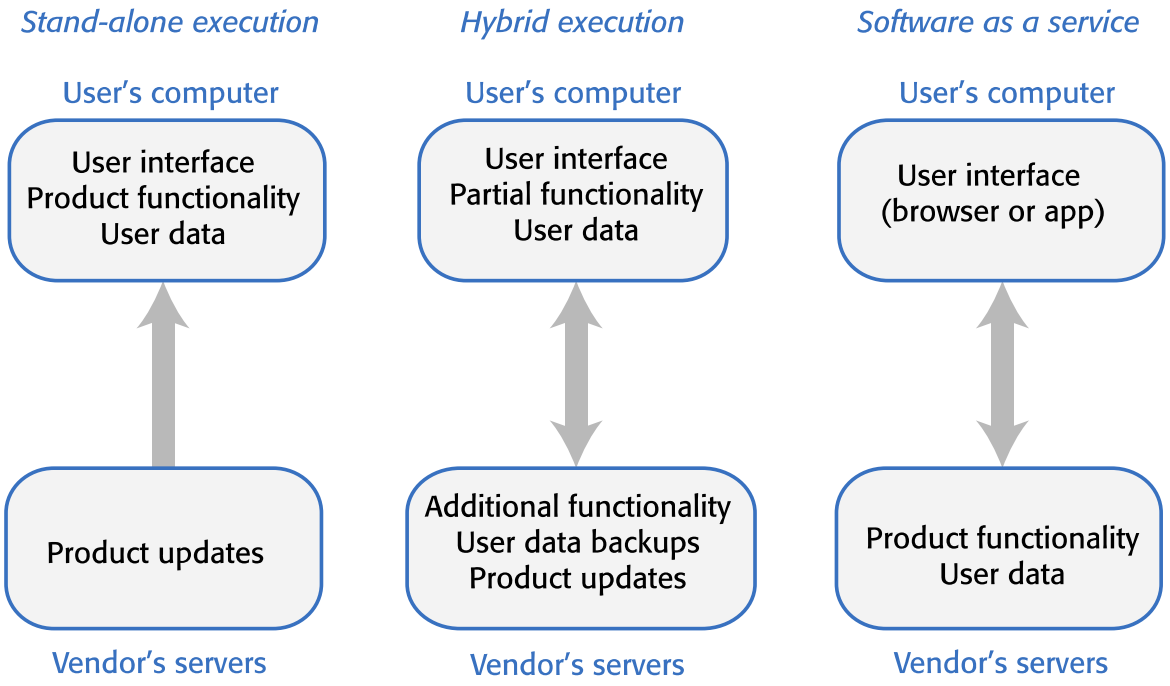
Table 1.1 Software product lines and platforms (2 of 2)

Technology	Description
Platform	A software (or software+hardware) product that includes functionality so that new applications can be built on it. An example of a platform that you probably use is Facebook. It provides an extensive set of product functionality but also provides support for creating “Facebook apps.” These add new features that may be used by a business or a Facebook interest group.

Software execution models

- **Stand-alone** The software executes entirely on the customer's computers.
- **Hybrid** Part of the software's functionality is implemented on the customer's computer but some features are implemented on the product developer's servers.
- **Software service** All of the product's features are implemented on the developer's servers and the customer accesses these through a browser or a mobile app.

Figure 1.3



Software execution models

Comparable software development

- The key feature of product development is that there is no external customer that generates requirements and pays for the software. This is also true for other types of software development:
  - ***Student projects*** Individuals or student groups develop software as part of their course. Given an assignment, they decide what features to include in the software.
  - ***Research software*** Researchers develop software to help them answer questions that are relevant to their research.
  - ***Internal tool development*** Software developers may develop tools to support their work - in essence, these are internal products that are not intended for customer release.

The product vision

- The starting point for software product development is a ‘product vision’.
- Product visions are simple statements that define the essence of the product to be developed.
- The product vision should answer three fundamental questions:
  - What is the product to be developed?
  - Who are the target customers and users?
  - Why should customers buy this product?

Moore's vision template

- FOR (target customer)
- WHO (statement of the need or opportunity)
- The (PRODUCT NAME) is a (product category)
- THAT (key benefit, compelling reason to buy)
- UNLIKE (primary competitive alternative)
- OUR PRODUCT (statement of primary differentiation)

Vision template example

“FOR a mid-sized company”s marketing and sales departments WHO need basic CRM functionality, THE CRM-Innovator is a Web-based service THAT provides sales tracking, lead generation, and sales representative support features that improve customer relationships at critical touch points. UNLIKE other services or package software products, OUR product provides very capable services at a moderate cost.”

Table 1.2 Information sources for developing a product vision (1 of 2)

Information source	Explanation
Domain experience	The product developers may work in a particular area (say,marketing and sales) and understand the software support that they need. They may be frustrated by the deficiencies in the software they use and see opportunities for an improved system.
Product experience	Users of existing software (such as word processing software) may see simpler and better ways of providing comparable functionality and propose a new system that implements this. New products can take advantage of recent technological developments such as speech interfaces.

Table 1.2 Information sources for developing a product vision (2 of 2)

Information source	Explanation
Customer experience	The software developers may have extensive discussions with prospective customers of the product to understand the problems that they face; constraints, such as interoperability, that limit their flexibility to buy new software; and critical attributes of the software that they need.
Prototyping and "playing around"	Developers may have an idea for software but need to develop a better understanding of that idea and what might be involved in developing it into a product. They may develop a prototype system as an experiment and "play around" with ideas and variations using that prototype system as a platform.

Example: A vision statement for the iLearn system

- FOR teachers and educators WHO need a way to help students use web-based learning resources and applications, THE iLearn system is an open learning environment THAT allows the set of resources used by classes and students to be easily configured for these students and classes by teachers themselves. UNLIKE Virtual Learning Environments, such as Moodle, the focus of iLearn is the learning process rather than the administration and management of materials, assessments and coursework. OUR product enables teachers to create subject and age-specific environments for their students using any web-based resources, such as videos, simulations and written materials that are appropriate.
- Schools and universities are the target customers for the iLearn system as it will significantly improve the learning experience of students at relatively low cost. It will collect and process learner analytics that will reduce the costs of progress tracking and reporting.

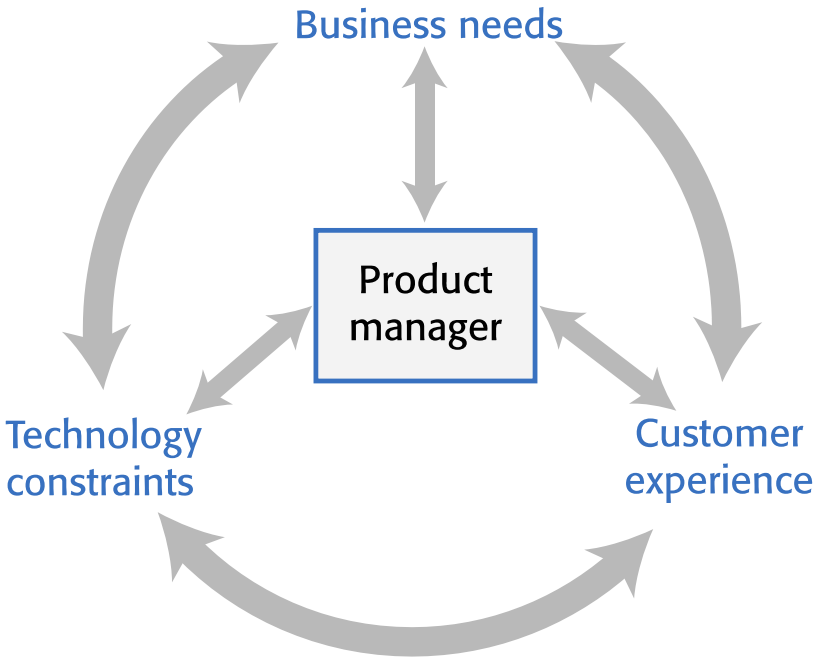
Software product management (1 of 2)

- Software product management is a business activity that focuses on the software products developed and sold by the business.
- Product managers (PMs) take overall responsibility for the product and are involved in planning, development and product marketing.

Software product management (2 of 2)

- Product managers are the interface between the organization, its customers and the software development team. They are involved at all stages of a product's lifetime from initial conception through to withdrawal of the product from the market.
- Product managers must look outward to customers and potential customers rather than focus on the software being developed.

Figure 1.4

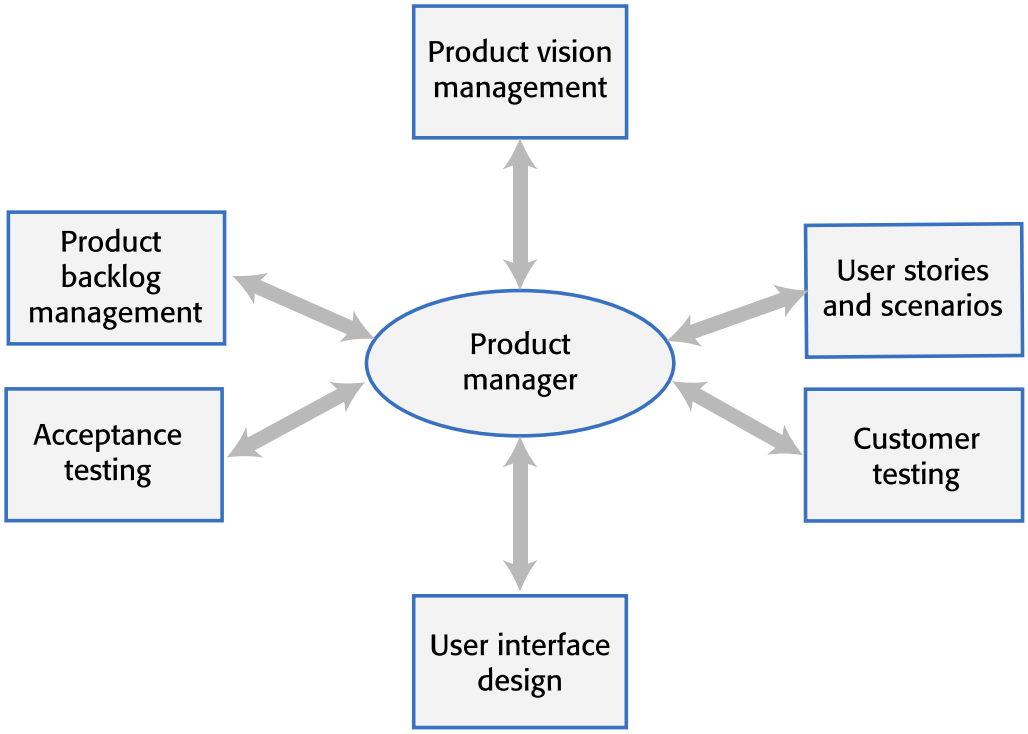


Product management concerns Business

Product management concerns

- ***Business needs*** PMs have to ensure that the software being developed meets the business goals of the software development company.
- ***Technology constraints*** PMs must make developers aware of technology issues that are important to customers.
- ***Customer experience*** PMs should be in regular contact with customers and potential customers to understand what they are looking for in a product, the types of users and their backgrounds and the ways that the product may be used.

Figure 1.5



Technical interactions of product managers

Technical interactions of product managers (1 of 4)

- Product vision management
  - The product manager may be responsible for helping with the development of the product vision. The should always be responsible for managing the vision, which involves assessing and evaluating proposed changes against the product vision. They should ensure that there is no ‘vision drift’
- Product roadmap development
  - A product roadmap is a plan for the development, release and marketing of the software. The PM should lead roadmap development and should be the ultimate authority in deciding if changes to the roadmap should be made.

Technical interactions of product managers (2 of 4)

- User story and scenario development
  - User stories and scenarios are used to refine a product vision and identify product features. Based on his or her knowledge of customers, the PM should lead the development of stories and scenarios.
- Product backlog creation and management
  - The product backlog is a prioritized ‘to-do’ list of what has to be developed. PMs should be involved in creating and refining the backlog and deciding on the priority of product features to be developed.

Technical interactions of product managers (3 of 4)

- Acceptance testing
  - Acceptance testing is the process of verifying that a software release meets the goals set out in the product roadmap and that the product is efficient and reliable. The PM should be involved in developing tests of the product features that reflect how customers use the product.
- Customer testing
  - Customer testing involves taking a release of a product to customers and getting feedback on the product's features, usability and business. PMs are involved in selecting customers to be involved in the customer testing process and working with them during that process.

Technical interactions of product managers (4 of 4)

- User interface design
  - Product managers should understand user limitations and act as surrogate users in their interactions with the development team. They should evaluate user interface features as they are developed to check that these features are not unnecessarily complex or force users to work in an unnatural way.

User Story and Scenario Development

- Used to refine vision into features and how the features should work.
- In natural language.
- PMs lead the scenarios and stories but takes input from the development team.
- Covered in chapter 3

Product Backlog Management

- To-do list.
- Added and refined during the development process.
- PMs are the authority on priority of the items in the backlog.
- PMs refine the items.
- Covered next week in chapter 2.

Acceptance Testing

- Verification the software meets goals set out in the roadmap and is efficient and reliable.
- PMs should be involved in this testing.
- Uses the scenarios to verify.
- Acceptance tests are used to verify before releasing to customers.

User Interface Design

- Critical.
- Challenging.
- PMs can act as surrogate users to evaluate the UI.
- PMs can arrange user testing.

Product prototyping (1 of 2)

- Product prototyping is the process of developing an early version of a product to test your ideas and to convince yourself and company funders that your product has real market potential.
  - You may be able to write an inspiring product vision, but your potential users can only really relate to your product when they see a working version of your software. They can point out what they like and don't like about it and make suggestions for new features.
  - A prototype may be also used to help identify fundamental software components or services and to test technology.

Product prototyping (2 of 2)

- Building a prototype should be the first thing that you do when developing a software product. Your aim should be to have a working version of your software that can be used to demonstrate its key features.
- You should always plan to throw-away the prototype after development and to re-implement the software, taking account of issues such as security and reliability.

Two-stage prototyping

- ***Feasibility demonstration*** You create an executable system that demonstrates the new ideas in your product. The aims at this stage are to see if your ideas actually work and to show funders and/or company management the original product features that are better than those in competing products.
- ***Customer demonstration*** You take an existing prototype created to demonstrate feasibility and extend this with your ideas for specific customer features and how these can be realized. Before you develop this type of prototype, you need to do some user studies and have a clearer idea of your potential users and scenarios of use.

Key points 1 (1 of 3)

- Software products are software systems that include general functionality that is likely to be useful to a wide range of customers.
- In product software engineering, the same company is responsible for deciding on the features that should be part of the product and the implementation of these features.

Key points 1 (2 of 3)

- Software products may be delivered as stand-alone systems running on the customer's computers, hybrid systems or service-based systems. In hybrid systems, some features are implemented locally and others are accessed over the Internet. All product features are remotely accessed in service-based products.
- A product vision should succinctly describe what is to be developed, who are the target customers for the product and why they should buy the product that you are developing.

Key points 1 (3 of 3)

- Domain experience, product experience, customer experience and an experimental software prototype may all contribute to the development of the product vision.

Key points 2

- Key responsibilities of product managers are product vision ownership, product roadmap development, creating user stories and the product backlog, customer and acceptance testing and user interface design.
- Product managers work at the interface between the business, the software development team and the product customers. They facilitate communications between these groups.
- You should always develop a product prototype to refine your own ideas and to demonstrate the planned product features to potential customers

Additional Readings

- http://www.productlineengineering.com/overview/what-is-ple.html
- https://www.joelonsoftware.com/2002/05/09/product-vision/
- https://www.romanpichler.com/blog/romans-product-management-framework/
- https://www.mindtheproduct.com/what-exactly-is-a-product-manager/

Project

- Project Status Report (PSR), Individual Status Report (ISR), Meeting Minutes templates
- Project proposal
- Meet and form teams

Next week

- Read all of chapters 1 & 2
  - Agile software engineering
- Create your semester project teams, start meeting
- Think about your semester product
- Assignment 1 due